

Java



Java Concepts Visual Guide



Understanding JVM, JRE, and JDK

Java has a **three-layer architecture** for execution:

1. **JDK (Java Development Kit)** – Used by developers to **write and compile** Java programs.
2. **JRE (Java Runtime Environment)** – Provides the necessary libraries and environment to **run Java applications**.
3. **JVM (Java Virtual Machine)** – The core component that **executes Java bytecode**.

◆ Think of it like this:

- **JDK = A full kitchen setup** (to cook food & serve it).
- **JRE = A microwave** (to reheat & serve food but not cook it).
- **JVM = The chef** (who actually prepares the food).



Java Virtual Machine (JVM)

◆ What is JVM?

- **JVM (Java Virtual Machine)** is the **engine** that executes Java programs.
- It is **platform-independent**, meaning Java can run on **any OS** as long as it has a compatible JVM.
- It **converts Java bytecode into machine code** that the OS understands.

◆ How JVM Works?

1 Java code is compiled into **bytecode** (`.class` files) using the **Java compiler** (`javac`).

2 JVM takes this **bytecode** and **interprets (or compiles)** it into **native machine code**.

3 The program runs on the **host system**.

◆ Key Features of JVM

- ✓ Platform-independent execution
- ✓ Automatic Memory Management (Garbage Collection)
- ✓ Security (Runs code inside a sandbox to prevent harmful operations)
- ✓ Supports Just-In-Time (JIT) Compilation for better performance

🔧 Java Runtime Environment (JRE)

◆ What is JRE?

- JRE (Java Runtime Environment) is **everything you need to run a Java program but NOT develop it**.
- It includes the **JVM + essential libraries** needed to execute Java applications.

◆ What's Inside the JRE?

- ◆ **JVM** (for executing Java code).
- ◆ **Core Java libraries** (for basic functionalities).
- ◆ **Runtime libraries** (like networking, UI, and database connectivity).

◆ Who Needs JRE?

- **End users** who only want to **run Java applications**.
- Example: If you download a **Java-based game**, your system needs JRE to run it.

💻 Java Development Kit (JDK)

◆ What is JDK?

- JDK (Java Development Kit) is **everything you need to develop, compile, and run Java programs**.

- It includes the **JRE + development tools (like compiler, debugger, and documentation tools)**.

◆ What's Inside the JDK?

- ◆ **JRE (includes JVM and Java libraries).**
- ◆ **Java Compiler (`javac`)** (compiles `.java` files into bytecode).
- ◆ **Debugger (`jdb`)** (used to debug Java programs).
- ◆ **JavaDocs (`javadoc`)** (generates documentation for Java code).

◆ Who Needs JDK?

- ✓ **Java Developers** – If you're writing Java code, you need the JDK.
- ✓ **Software Engineers** – Anyone working with Java-based projects.
- ✓ **Students & Learners** – If you're learning Java, install the JDK to write and run programs.

Class and Object

◆ Definition

- **Class:** A class in Java is a **blueprint or template** that defines the attributes (**state**) and behaviors (**methods**) of objects. It acts as a **mold** from which multiple objects can be created.
- **Object:** An object is an **instance of a class** that represents a real-world entity. Each object has its **own state (variables) and behavior (methods)**.

◆ Key Points

- ✓ A **class** defines the structure, but an **object** is the actual instance with values.
- ✓ Objects **store data in fields (variables)** and **use methods (functions)** to perform actions.
- ✓ Objects are **created dynamically at runtime** using the `new` keyword.

◆ Characteristics of a Class

1. **Encapsulation** – Wrapping data and behavior together.

2. **Abstraction** – Hiding implementation details.
3. **Inheritance** – Acquiring properties from another class.
4. **Polymorphism** – Taking multiple forms in behavior.

Object-Oriented Programming (OOP)

◆ Definition

- **Object-Oriented Programming (OOP)** is a programming paradigm based on the concept of **objects**, which bundle **data and behavior** together.
- It focuses on designing software using **reusable and modular components**.

◆ Principles of OOP

1. **Encapsulation** – Hides data using access modifiers and provides controlled access through getters & setters.
2. **Abstraction** – Hides unnecessary details and exposes only essential features.
3. **Inheritance** – Allows a class to **inherit properties and methods** from another class.
4. **Polymorphism** – Enables the same method name to have different behaviors.
 - **Method Overloading** (Compile-time Polymorphism) – Same method name with different parameters.
 - **Method Overriding** (Runtime Polymorphism) – Child class provides a different implementation of a method from the parent class.

◆ Advantages of OOP

- ✓ **Modularity** – Code is organized into separate objects, making it easier to manage.
- ✓ **Code Reusability** – Inheritance reduces redundancy.
- ✓ **Scalability** – Objects can be modified independently without affecting the entire system.
- ✓ **Security** – Encapsulation protects data by restricting direct access.



Method Overloading in Java

◆ What is Method Overloading?

- **Method Overloading** means defining multiple methods **with the same name but different parameters**.
- The **compiler decides which method to call based on the arguments passed**.
- It improves **code readability** and allows **flexibility** in using the same method for different inputs.



Key Rules for Method Overloading



Methods **must have the same name** but **different parameters**.



Difference can be in:

- **Number of parameters** (e.g., `add(int, int)` vs. `add(int, int, int)`).
- **Data types of parameters** (e.g., `multiply(int, int)` vs. `multiply(double, double)`).
- **Order of parameters** (e.g., `print(int, String)` vs. `print(String, int)`).



Return type does NOT matter (You **cannot overload based on return type alone**).



Method overloading is resolved at compile-time (Compile-time Polymorphism).



Example: Adding Two and Three Numbers

```
class Calculator {  
    int add(int a, int b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
}
```



Output

```
Sum of 2 numbers: 15  
Sum of 3 numbers: 30
```



Memory Management in Java

In Java, memory is divided into two main areas:

1 **Stack Memory** – Stores method calls and local variables.

2 **Heap Memory** – Stores objects and instance variables.

Each part of memory has a specific role and is managed by **JVM (Java Virtual Machine)**.

◆ Stack Memory (Method Execution & Local Variables)

- Stack memory is **used for execution of methods**.
- Every method call creates a **new block** in the stack.
- When a method **finishes execution**, its stack frame is **removed (LIFO - Last In, First Out)**.
- Stores **local variables** and **object references**, but **not actual objects**.

Characteristics:

- ✓ Fast memory access (as it follows **LIFO order**).
- ✓ Each thread gets its own **separate stack memory**.
- ✓ **Variables inside a method exist only while the method is running**.

◆ Heap Memory (Object Storage)

- Heap memory is used for **storing objects** in Java.
- Whenever you create an object using `new`, it is stored in the **heap**.
- **All objects persist in heap memory until Garbage Collection (GC) removes unused objects**.

Characteristics:

- ✓ Slower access than stack (as it stores objects dynamically).
- ✓ **All objects are stored in heap memory, and multiple methods can access them using references**.
- ✓ Shared across all threads (**unlike stack memory, which is thread-specific**).

Example: How Stack and Heap Work

```
class Calculator {
    int var; // Instance variable stored in heap memory

    // Method to add two numbers
    int add(int a, int b) {
        int sum = a + b; // 'sum' is stored in stack memory
        return sum;
    }
}

public class Demo {
    public static void main(String[] args) {
        // Creating two objects of Calculator
        Calculator obj1 = new Calculator(); // Stored in heap
        Calculator obj2 = new Calculator(); // Stored in heap

        // Assigning values
        obj1.var = 1; // obj1's 'var' is 1
        obj2.var = 10; // obj2's 'var' is 10

        System.out.println("obj1.var: " + obj1.var);
        System.out.println("obj2.var: " + obj2.var);

        // Calling add method
        int result1 = obj1.add(5, 10);
        int result2 = obj2.add(20, 30);

        System.out.println("Sum using obj1: " + result1);
        System.out.println("Sum using obj2: " + result2);
    }
}
```

How Does Stack and Heap Work in This Code?

Step 1: When `main()` Starts

📌 Stack is created for `main()`, and JVM starts execution.

Step 2: Object Creation (`obj1` and `obj2`)

- **Heap Memory:**
 - `obj1` and `obj2` are stored in **heap memory**.
 - Each object has **its own copy of instance variables** (`var`).

Step 3: Assigning Values (`obj1.var = 1; obj2.var = 10;`)

- `obj1.var` is updated to 1.
- `obj2.var` is updated to 10.
- **Since objects have separate memory in the heap, changing one object's variable does NOT affect the other.**

Step 4: Calling `add()` Method

- When `obj1.add(5, 10)` is called:
 - **Stack Memory:** Creates a stack frame for `add()`, and `a`, `b`, and `sum` are stored here.
 - **Heap Memory:** The object (`obj1`) remains in heap.
 - Once the method finishes, the `sum` variable is removed from the stack.
- When `obj2.add(20, 30)` is called:
 - A **new stack frame** is created for `add()` with different values.
 - The result is returned, and the frame is removed.

Step 5: Printing Values

- Since **each object has its own memory in heap**, `obj1.var` and `obj2.var` have **independent values**.

String in Java

◆ What is a String?

- A **String** in Java is a **sequence of characters** stored as an object of the `String` class.
- Java Strings are **immutable**, meaning their values cannot be changed after creation.

◆ String Immutability

- Once a String object is created, its value **cannot be modified**.
- If we change a String, a **new object** is created in memory, and the reference is updated.

Example:

```
String str1 = "Hello";  
str1 = str1 + " World";  
System.out.println(str1); // Output: "Hello World"
```

- The original `"Hello"` string is not modified; instead, a **new String object** `"Hello World"` is created.

◆ String Constant Pool

- Java maintains a **String Pool (intern pool)** inside the heap to optimize memory usage.
- If a String is **created using a string literal**, it is stored in the **String Pool**.
- If a String is **created using `new` keyword**, it is stored in **heap memory**.

Example:

```
String s1 = "Java"; // Stored in String pool  
String s2 = "Java"; // Reuses same reference from the pool  
String s3 = new String("Java"); // Creates a new object in heap  
System.out.println(s1 == s2); // true (same reference in pool)  
System.out.println(s1 == s3); // false (different memory locations)
```

◆ Mutable vs Immutable Strings

- **Immutable:** `String` class (modifying creates a new object).
- **Mutable:** `StringBuilder` and `StringBuffer` classes (modifying does not create new objects).

Example of Mutable Strings:

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: "Hello World" (modified in same object)
```

◆ When to Use Which?

- Use **String** when immutability is required (e.g., storing constants, keys in maps).
- Use **StringBuilder** or **StringBuffer** when frequent modifications are needed (e.g., loops, concatenations).

StringBuffer vs StringBuilder

💡 We will learn about threading later on so you can skip this part for now if you really want to or you can just go through it and remember that something like this was here.

◆ StringBuffer

- `StringBuffer` is a **mutable** class used to modify strings without creating new objects.
- It is **thread-safe** and synchronized, meaning multiple threads can safely access it.
- Because of synchronization, it is **slower** than `StringBuilder`.

Example:

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: "Hello World"
```

◆ StringBuilder

- `StringBuilder` is also mutable and allows modification of strings.
- It is **not thread-safe**, meaning it does not synchronize methods.
- Faster than `StringBuffer` due to lack of synchronization.

Example:

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: "Hello World"
```

◆ Key Differences

Feature	StringBuffer	StringBuilder
Mutable?	Yes	Yes
Thread-safe?	Yes	No
Synchronized?	Yes	No
Performance	Slower (due to sync)	Faster

◆ When to Use Which?

- Use **StringBuffer** when working with **multi-threaded** applications where thread safety is required.
- Use **StringBuilder** when working in a **single-threaded** environment where performance is critical.

Static Block and Class.forName() in Java

◆ Static Block

- A **static block** in Java is used for **static initialization**.
- It is executed **only once**, when the class is **loaded into memory**.
- It is mainly used for **initializing static variables or performing setup operations**.

Example:

```
class Example {  
    static {  
        System.out.println("Static block executed!");  
    }  
    public static void main(String[] args) {  
        System.out.println("Main method executed!");  
    }  
}
```

Output:

```
Static block executed!  
Main method executed!
```

◆ `Class.forName()` Method

- `Class.forName("className")` is used to **dynamically load a class at runtime**.
- It is commonly used in **JDBC (Java Database Connectivity)** to load database drivers.
- It triggers the execution of **static blocks** of the loaded class.

Example:

```
class Demo {  
    static {
```

```

        System.out.println("Demo class loaded!");
    }
}

public class Test {
    public static void main(String[] args) throws ClassNotFoundException {
        Class.forName("Demo"); // Dynamically loads Demo class
    }
}

```

Output:

```
Demo class loaded!
```

◆ When to Use `Class.forName()` ?

- ✓ **JDBC:** Loading database drivers dynamically.
- ✓ **Reflection:** Dynamically loading classes without knowing their names at compile time.
- ✓ **Plugin Systems:** Loading modules dynamically based on user input.

Encapsulation, Getters & Setters in Java

◆ What is Encapsulation?

- **Encapsulation** is the technique of **binding data (variables) and methods (functions) together** within a class.
- It **hides the internal details** of how an object works and only exposes what is necessary.
- It helps in **data security and integrity** by restricting direct access to fields.

◆ Getters and Setters

- Getters (`getMethod()`) allow controlled access to private variables.
- Setters (`setMethod()`) provide a controlled way to modify values.

Example of Encapsulation:

```
class Person {  
    private String name; // Private field  
  
    // Getter method  
    public String getName() {  
        return name;  
    }  
  
    // Setter method  
    public void setName(String name) {  
        this.name = name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.setName("John");  
        System.out.println(p.getName()); // Output: John  
    }  
}
```

◆ Why Use Getters and Setters Instead of Public Fields?

- **Encapsulation & Security:** Protects data from unauthorized access.
- **Validation:** We can apply **input validation** inside setter methods.
- **Read-Only or Write-Only Fields:** We can create **read-only** (getter only) or **write-only** (setter only) fields.
- **Flexibility:** Allows changing the internal implementation **without affecting other parts of the code**.

◆ Real-World Example

Bank Account System:

```
class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Invalid deposit amount");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(500);
        System.out.println("Balance: " + account.getBalance()); // Output: Balance: 500.0
    }
}
```

◆ Key Takeaway

💡 Q: If we have both getter and setter, why not just make the field public?

1. **We lose control over data modification.**
2. **Validation cannot be applied.**
3. **Security risks arise.**
4. **Future modifications become harder.**

Constructors in Java

◆ What is a Constructor?

- A **constructor** is a special method that is called when an object is created.
- It is used to **initialize objects**.
- A constructor **has the same name as the class** and does **not have a return type**.

◆ Types of Constructors

1 Default Constructor

- A **default constructor** is automatically provided by Java if no constructor is explicitly defined.
- It initializes objects with **default values** (e.g., `null` for objects, `0` for numbers, `false` for boolean values).

2 Parameterized Constructor

- A **parameterized constructor** takes arguments to **set initial values for an object**.

Example:

```
class Car {  
    String model;  
    int year;  
  
    // Parameterized constructor  
    Car(String m, int y) {  
        model = m;  
        year = y;  
    }  
  
    void display() {  
        System.out.println("Model: " + model + ", Year: " + year);  
    }  
}
```



```

    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car("Toyota", 2022);
        car1.display(); // Output: Model: Toyota, Year: 2022
    }
}

```

◆ Why Use a Constructor Instead of Direct Field Assignment?

- ✓ **Encapsulation:** The constructor ensures that all necessary values are set when the object is created.
- ✓ **Code Reusability:** Reduces repetitive code by setting up object properties in one place.
- ✓ **Readability & Maintainability:** Improves the structure of the code, making it easier to understand.
- ✓ **Consistency:** Ensures that an object cannot be created in an invalid state.
- ✓ **Preprocessing & Validation:** Allows validation of inputs before assigning values.

Example: Why Constructor is Preferred Over Direct Field Assignment

```

class Person {
    String name;
    int age;

    // Constructor ensures valid values are assigned
    Person(String name, int age) {
        if (age <= 0) {
            throw new IllegalArgumentException("Age must be positive");
        }
        this.name = name;
    }
}

```

```

        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object with valid data
        Person p1 = new Person("John", 25);
        p1.display(); // Output: Name: John, Age: 25
    }
}

```

◆ What If We Used Direct Field Assignment?

```

class Person {
    String name;
    int age;
}

public class Main {
    public static void main(String[] args) {
        Person p1 = new Person();
        p1.name = "John";
        p1.age = -5; // Invalid age, but no validation!
        System.out.println("Name: " + p1.name + ", Age: " + p1.age);
    }
}

```

✗ Problem: Direct field assignment does not enforce validation, allowing incorrect data entry.

Inheritance in Java

◆ What is Inheritance?

- **Inheritance** is a fundamental **OOP (Object-Oriented Programming)** concept that allows a class (**child/subclass**) to **acquire the properties and behavior** of another class (**parent/superclass**).
- It enables **code reuse** and promotes **hierarchical classification**.

◆ Need for Inheritance

✓ **Code Reusability:** Eliminates code duplication by allowing child classes to reuse the code from parent classes.

✓ **Extensibility:** Helps in extending the functionality of existing classes without modifying them.

✓ **Maintainability:** Reduces redundancy and makes the code more maintainable.

✓ **Hierarchy Representation:** Represents relationships between different classes efficiently.

◆ Types of Inheritance in Java

1 Single Inheritance

- When a class **inherits from a single superclass**.
- Example:

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // Inherited method
        d.bark();
    }
}

```

2 Multilevel Inheritance

- When a class **inherits from another derived class**.
- Example:

```

class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}
class Mammal extends Animal {
    void walk() {
        System.out.println("Mammals walk.");
    }
}
class Dog extends Mammal {
    void bark() {
        System.out.println("Dog barks.");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // Inherited from Animal
        d.walk(); // Inherited from Mammal
        d.bark(); // Defined in Dog
    }
}

```

```
}  
}
```

3 Multiple Inheritance (via Interfaces)

- **Java does not support multiple inheritance with classes** to avoid ambiguity.
- However, **multiple inheritance can be achieved using interfaces**.
- Example:

```
interface Animal {  
    void eat();  
}  
interface Pet {  
    void play();  
}  
class Dog implements Animal, Pet {  
    public void eat() {  
        System.out.println("Dog eats food.");  
    }  
    public void play() {  
        System.out.println("Dog plays fetch.");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.play();  
    }  
}
```

this() and super() Constructors in Java

◆ super() Constructor

- In Java, **every constructor calls the constructor of its parent class** using `super()`, either **explicitly or implicitly**.
- If no `super()` is mentioned, Java **automatically calls the no-argument constructor of the parent class**.
- It must be the **first statement** inside a constructor.

Example: Implicit `super()` Call

```
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }
}
class Dog extends Animal {
    Dog() {
        // super() is implicitly called here
        System.out.println("Dog constructor called");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Output:

```
Animal constructor called
Dog constructor called
```

Example: Explicit `super()` Call with Parameters

```
class Animal {
    Animal(String type) {
```

```

        System.out.println("Animal Type: " + type);
    }
}
class Dog extends Animal {
    Dog(String breed) {
        super("Mammal"); // Explicitly calling parent constructor
        System.out.println("Dog Breed: " + breed);
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog("Labrador");
    }
}

```

Output:

```

Animal Type: Mammal
Dog Breed: Labrador

```

◆ **this()** Constructor

- The **this()** constructor is used **to call another constructor within the same class**.
- It helps in **constructor chaining**, where one constructor calls another.
- Like **super()**, it **must be the first statement** in a constructor.

Example: Constructor Chaining Using **this()**

```

class Car {
    String model;
    int year;

    // Constructor with default values

```

```

Car() {
    this("Unknown", 2000); // Calls parameterized constructor
}

// Parameterized constructor
Car(String model, int year) {
    this.model = model;
    this.year = year;
}

void display() {
    System.out.println("Model: " + model + ", Year: " + year);
}
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car(); // Calls the no-argument constructor
        car1.display();

        Car car2 = new Car("Toyota", 2023);
        car2.display();
    }
}

```

Output:

```

Model: Unknown, Year: 2000
Model: Toyota, Year: 2023

```

◆ Key Differences Between `this()` and `super()`

Feature	<code>this()</code>	<code>super()</code>
Refers to	Current class constructor	Parent class constructor

Used for	Calling another constructor within the same class	Calling a constructor from the parent class
Must be first statement?	Yes	Yes

◆ When to Use `this()` and `super()` ?

- ✓ Use `this()` when you want to **reuse a constructor** inside the same class.
- ✓ Use `super()` when you want to **call a constructor of the parent class**.
- ✓ Both `this()` and `super()` **cannot be used together in the same constructor**.
- ✓ Use constructor chaining with `this()` to set default values efficiently.

Method Overriding in Java

◆ What is Method Overriding?

- **Method Overriding** is a feature in Java that allows a **subclass** to provide a specific implementation of a method that is already defined in its **superclass**.
- The method in the subclass must have the **same name, return type, and parameters** as the method in the superclass.
- It enables **runtime polymorphism**, where method calls are resolved dynamically at runtime.

◆ Rules for Method Overriding

- ✓ The method **must have the same name and parameters** as the parent class method.
- ✓ The **return type must be the same** (or a subclass of the return type in Java 5+).
- ✓ The method **cannot have a more restrictive access modifier** than the overridden method.
- ✓ The method **must be inherited** by the subclass (i.e., cannot override `private`, `static`, or `final` methods).

✓ **Overridden methods in the subclass should have equal or broader access levels** (e.g., `protected` → `public`).

◆ Example of Method Overriding

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myPet = new Dog(); // Runtime polymorphism
        myPet.makeSound(); // Output: Dog barks
    }
}
```

◆ `@Override` Annotation

- The `@Override` annotation **ensures that a method is actually overriding a method in the parent class.**
- If there is a mismatch (e.g., wrong method name, incorrect parameters), the compiler will generate an error.

Example with `@Override` Error Prevention:

```

class Parent {
    void show() {
        System.out.println("Parent class");
    }
}

class Child extends Parent {
    @Override
    void show(int num) { // ❌ Compilation Error - Parameters don't match
        System.out.println("Child class");
    }
}

```

◆ Calling Parent Class Method Using **super**

- If the overridden method needs to **invoke the parent class's version**, `super.methodName()` can be used.

Example:

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        super.makeSound(); // Calls parent class method
        System.out.println("Dog barks");
    }
}

public class Main {

```

```

public static void main(String[] args) {
    Dog d = new Dog();
    d.makeSound();
}
}

```

Output:

```

Animal makes a sound
Dog barks

```

◆ Difference Between Method Overloading and Method Overriding

Feature	Method Overloading	Method Overriding
Definition	Multiple methods with same name but different parameters	A subclass provides a new implementation of a method from the superclass
Parameters	Must be different	Must be the same
Return Type	Can be different	Must be the same (or covariant)
Access Modifier	Can have any access modifier	Cannot be more restrictive than the overridden method
Static Methods	Can be overloaded	Cannot be overridden
Final Methods	Can be overloaded	Cannot be overridden
Occurs in	Same class	Parent-child relationship

◆ When to Use Method Overriding?

- ✓ When we want to **customize behavior in a subclass** without changing the superclass.
- ✓ When we implement **runtime polymorphism** to decide which method is called dynamically at runtime.
- ✓ When we need to **maintain a contract** (e.g., overriding methods from an interface or abstract class).

- ✓ When we need **specialized behavior** in a subclass that differs from the generic behavior of a superclass.

Access Modifiers

	Private	Protected	Public	Default
Same class	Yes	Yes	Yes	Yes
Same package subclass	NO	Yes	Yes	Yes
Same package non-subclass	NO	Yes	Yes	Yes
Different package subclass	NO	Yes	Yes	NO
Different package non-subclass	NO	NO	Yes	NO

☕ Polymorphism in Java

◆ What is Polymorphism?

- **Polymorphism** means "**many forms**" and allows a single interface (method or class) to be used for different types of data.
- It enables **code reusability, flexibility, and scalability**.
- There are **two types of polymorphism** in Java:
 1. **Compile-time Polymorphism (Method Overloading)**
 2. **Runtime Polymorphism (Method Overriding)**

◆ 1 Compile-time Polymorphism (Method Overloading)

- Also known as **static polymorphism**.

- Method overloading allows multiple methods with **the same name but different parameters** within the same class.
- The compiler determines which method to call based on the method signature.

Example of Method Overloading:

```
class MathUtils {
    // Method with two parameters
    int add(int a, int b) {
        return a + b;
    }

    // Method with three parameters (Overloaded)
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        MathUtils obj = new MathUtils();
        System.out.println(obj.add(10, 20));    // Output: 30
        System.out.println(obj.add(10, 20, 30)); // Output: 60
    }
}
```

◆ 2 Runtime Polymorphism (Method Overriding)

- Also known as **dynamic polymorphism**.
- Method overriding occurs when a subclass provides a **specific implementation** of a method defined in its superclass.
- The method to be executed is determined at **runtime** based on the actual object type.

Example of Method Overriding:

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myPet = new Dog(); // Runtime polymorphism
        myPet.makeSound(); // Output: Dog barks
    }
}

```

◆ Differences Between Compile-time and Runtime Polymorphism

Feature	Compile-time Polymorphism (Method Overloading)	Runtime Polymorphism (Method Overriding)
Definition	Multiple methods with the same name but different parameters	Subclass provides a specific implementation of a parent class method
Method Resolution	Resolved at compile-time	Resolved at runtime
Method Signature	Methods must have different parameters	Methods must have the same parameters
Return Type	Can be different	Must be the same (or covariant)

Access Modifier	Can be anything	Cannot be more restrictive than the overridden method
Static Methods	Can be overloaded	Cannot be overridden
Final Methods	Can be overloaded	Cannot be overridden

◆ Real-World Example of Polymorphism

Scenario: Payment System

Imagine an **online payment system** where different payment methods (Credit Card, PayPal, UPI) process payments differently but are called using the same method.

```
class Payment {
    void processPayment() {
        System.out.println("Processing generic payment");
    }
}

class CreditCard extends Payment {
    @Override
    void processPayment() {
        System.out.println("Processing Credit Card Payment");
    }
}

class PayPal extends Payment {
    @Override
    void processPayment() {
        System.out.println("Processing PayPal Payment");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment payment1 = new CreditCard();
    }
}
```



```
payment1.processPayment(); // Output: Processing Credit Card Payment
```

```
Payment payment2 = new PayPal();
```

```
payment2.processPayment(); // Output: Processing PayPal Payment
```

```
}
```

```
}
```

◆ When to Use Polymorphism?

- ✓ When **multiple classes share a common behavior** but implement it differently.
- ✓ When we need **code flexibility** and reusability.
- ✓ When implementing **frameworks and APIs** to provide a standard interface with customizable behavior.
- ✓ When working with **collections, design patterns, and dependency injection** in Java.



Dynamic Method Dispatch (Runtime Polymorphism)

◆ What is Dynamic Method Dispatch?

- **Dynamic Method Dispatch (DMD)** is the mechanism by which a **method call to an overridden method is resolved at runtime** rather than compile-time.
- It enables **runtime polymorphism**, where the decision of which method to invoke depends on the **actual object type**, not the reference type.
- The **overridden method in the subclass is executed**, even if the reference variable belongs to the superclass.

◆ How Does It Work?

- ✓ A **reference variable** of the parent class is used to refer to a **child class object**.
- ✓ The overridden method of the **actual object's class** is called at runtime.
- ✓ **JVM determines the method to call dynamically, based on the object type.**

◆ Example of Dynamic Method Dispatch

```
class Parent {  
    void show() {  
        System.out.println("Parent class method");  
    }  
}  
class Child extends Parent {  
    @Override  
    void show() {  
        System.out.println("Child class method");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Parent obj = new Child(); // Parent reference, Child object  
        obj.show(); // Output: Child class method  
    }  
}
```

◆ Why is Dynamic Method Dispatch Useful?

- ✓ **Achieves Runtime Polymorphism** – Allows execution of overridden methods dynamically.
- ✓ **Enhances Code Reusability** – Parent class references can hold any subclass object.
- ✓ **Supports Frameworks & APIs** – Java libraries rely heavily on method dispatch.
- ✓ **Improves Maintainability** – Changes to a subclass don't require modifying calling code.

◆ Example: Dynamic Method Dispatch in a Real-world Scenario

Imagine a **drawing application** that allows users to create different shapes (Circle, Rectangle, etc.). Each shape has a `draw()` method, but the implementation is

different for each shape.

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}
class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
public class Main {
    public static void main(String[] args) {
        Shape shape;
        shape = new Circle();
        shape.draw(); // Output: Drawing a Circle

        shape = new Rectangle();
        shape.draw(); // Output: Drawing a Rectangle
    }
}
```

◆ Important Rules for Dynamic Method Dispatch

- ✓ Only instance methods (non-static) can be overridden and resolved dynamically.
- ✓ Static methods are not dynamically dispatched; they are resolved at compile-

time.

✓ **Private and final methods cannot be overridden**, so they are not part of dynamic dispatch.

✓ **Constructors are not overridden**, so they don't participate in method dispatch.

◆ Key Differences Between Static and Dynamic Method Resolution

Feature	Static Method Resolution	Dynamic Method Dispatch
Determination Time	Compile-time	Runtime
Overridden Methods	Not applicable	Applicable
Method Type	Static methods	Instance methods
Execution Control	Compiler decides	JVM decides at runtime
Example Mechanism	Method Overloading	Method Overriding

◆ When to Use Dynamic Method Dispatch?

✓ When working with **frameworks and APIs** that depend on runtime polymorphism.

✓ When designing **extensible applications** that allow future modifications with minimal changes.

✓ When implementing **interfaces and abstract classes**, enabling flexible architecture.

✓ When needing **real-world behavior modeling**, such as GUI event handling or plugin-based systems.

Upcasting and Downcasting in Java

◆ What is Upcasting?

- **Upcasting** is the process of assigning a **subclass object to a superclass reference**.
- It is **implicit** and does not require explicit type casting.
- Upcasting is useful in **dynamic method dispatch (runtime polymorphism)**.

Example of Upcasting:

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Upcasting (implicit)
        a.makeSound(); // Output: Dog barks (Runtime Polymorphism)
    }
}
```

◆ Benefits of Upcasting

- ✓ **Code Flexibility** – A superclass reference can hold any subclass object.
- ✓ **Dynamic Method Dispatch** – Allows calling overridden methods dynamically.
- ✓ **Extensibility** – Supports frameworks, APIs, and dependency injection.
- ✓ **Loose Coupling** – Reduces dependencies between tightly coupled classes.

◆ What is Downcasting?

- **Downcasting** is the process of converting a **superclass reference back to a subclass reference**.
- It **requires explicit casting** and is **unsafe** if not handled correctly.
- Downcasting is used when we need access to **subclass-specific methods**.

Example of Downcasting:

```

class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Upcasting
        Dog d = (Dog) a; // Downcasting (explicit)
        d.bark(); // Output: Dog barks
    }
}

```

◆ When Do We Need Downcasting?

- ✓ When accessing **subclass-specific methods** that are not present in the superclass.
- ✓ When working with **collections that store superclass references**.
- ✓ When handling **generic types in Java frameworks**.

◆ Why Use Upcasting Even When We Inherit?

Even though a subclass **inherits** all methods from the superclass, upcasting is still needed for:

1. **Runtime Polymorphism** – The correct overridden method is resolved at runtime.
2. **Working with Collections** – Collections often store superclass references.
3. **Interfacing with APIs** – Many Java APIs expect superclass references.

4. **Method Generalization** – Methods designed to work with superclass references can handle multiple subclass types.

Example: Why Upcasting is Required in Inheritance?

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starting");
    }
}
class Car extends Vehicle {
    void start() {
        System.out.println("Car starting");
    }
    void drive() {
        System.out.println("Car is driving");
    }
}
public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car(); // Upcasting (even though Car extends Vehicle)
        v.start(); // Output: Car starting (runtime polymorphism)

        // v.drive(); // ❌ Compilation Error: drive() is not available in Vehicle
    }
}
```

◆ Key Differences Between Upcasting and Downcasting

Feature	Upcasting	Downcasting
Definition	Assigning subclass object to superclass reference	Converting superclass reference back to subclass reference
Explicit Casting	Not required (implicit)	Required (explicit)
Access to Subclass Methods	Not available	Available after casting

Polymorphism	Supports runtime polymorphism	Used when subclass-specific methods are needed
Safety	Always safe	Risk of <code>ClassCastException</code> if not properly checked

◆ When to Use Upcasting and Downcasting?

- ✓ **Use Upcasting** when working with **dynamic method dispatch, frameworks, APIs, and generic collections**.
- ✓ **Use Downcasting** only when **necessary to access subclass-specific methods**, ensuring proper type checking.
- ✓ **Always check type compatibility** before performing downcasting using `instanceof`.

Safe Downcasting Using `instanceof`

```
class Animal {}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        if (a instanceof Dog) {
            Dog d = (Dog) a; // Safe Downcasting
            d.bark();
        }
    }
}
```

- ✓ **Ensures safety by checking the actual object type before downcasting.**

Interfaces in Java

◆ What is an Interface?

- An **interface** in Java is a **blueprint** for a class that defines a **set of abstract methods** (methods without a body).
- A class that **implements** an interface must **provide implementations** for all its methods.
- Interfaces help achieve **abstraction** and **multiple inheritance** in Java.

◆ Syntax of an Interface

```
interface Animal {  
    void makeSound(); // Abstract method (no body)  
    int AGE = 10;     // public static final by default  
}  
  
class Dog implements Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("Woof Woof");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal dog = new Dog();  
        dog.makeSound(); // Output: Woof Woof  
        System.out.println("Default age: " + Animal.AGE); // Access constant  
    }  
}
```

◆ Key Features of Interfaces

- ✓ **Supports Multiple Inheritance** – A class can implement multiple interfaces.
- ✓ **100% Abstraction** – Only method signatures are defined, no implementation (prior to Java 8).
- ✓ **Loose Coupling** – Reduces dependencies between classes.
- ✓ **Default Methods (Java 8+)** – Interfaces can have default implementations.
- ✓ **Static Methods (Java 8+)** – Interfaces can contain static methods.
- ✓ **Private Methods (Java 9+)** – Interfaces can have private methods for code reuse.

◆ Multiple Interface Implementation

```
interface Swimmer {
    void swim();
}

interface Runner {
    void run();
}

class Athlete implements Swimmer, Runner {
    @Override
    public void swim() {
        System.out.println("Athlete is swimming");
    }

    @Override
    public void run() {
        System.out.println("Athlete is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Athlete athlete = new Athlete();
        athlete.swim(); // Output: Athlete is swimming
    }
}
```

```

        athlete.run(); // Output: Athlete is running
    }
}

```

◆ Default Methods in Interfaces (Java 8+)

```

interface Vehicle {
    void start();

    // Default method with implementation
    default void honk() {
        System.out.println("Beep Beep!");
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }
    // No need to implement honk() as it has a default implementation
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start(); // Output: Car started
        car.honk(); // Output: Beep Beep!
    }
}

```

◆ Interface vs Abstract Class

Feature	Interface	Abstract Class
---------	-----------	----------------

Methods	Abstract, default, static, private	Abstract & concrete methods
Multiple Inheritance	Supports multiple inheritance	Does not support multiple inheritance
Access Modifiers	Methods are public by default	Can have protected, private, public methods
Fields	Only <code>public static final</code> constants	Can have instance variables
Constructor	No constructor	Can have constructors
Implementation	0-100% implementation	0-100% implementation

◆ Functional Interfaces (Java 8+)

- A **functional interface** is an interface with **exactly one abstract method**.
- Used with Lambda expressions to enable functional programming in Java.

```
@FunctionalInterface
interface Greeting {
    void greet(String name);
}

public class Main {
    public static void main(String[] args) {
        // Implementation using lambda expression
        Greeting greeting = (name) → System.out.println("Hello, " + name + "!");

        greeting.greet("John"); // Output: Hello, John!
    }
}
```

◆ When to Use an Interface?

- ✓ When multiple classes need to share a **common behavior** but do not share a common state.

- ✓ When you want to **ensure method standardization** across different implementations.
- ✓ When implementing **dependency injection** in a modular and testable application.
- ✓ When you want **multiple inheritance**, since Java allows a class to implement multiple interfaces but not extend multiple classes.
- ✓ When creating **API contracts** to provide a stable interface for client code.